Binding Approval to Dispatch in Tool-Using Agents: A Reproducible Lifecycle Conformance Study

Rajeshkumar Sampathrajan
Independent Researcher

***Abstract*—An agent host can inspect a tool descriptor and still dispatch an operation that differs from the operation it approved. Catalogue updates, cross-server tool-name collisions, connection replacement and mutable call arguments create distinct opportunities for this divergence. We present a reproducible conformance study of this lifecycle boundary in authored local adapters modeling a Model Context Protocol style host. The artifact separates a pinned metadata scanner from six explicitly authored host adapters and evaluates actual sink receipts against a separately implemented structural oracle. It combines 36 targeted scenarios, all 259 event sequences of length at most three over a declared six-event model, deterministic concurrency checks and local paired timing samples. Both the revocation-aware snapshot control and the per-binding generation gate satisfy all 36 targeted scenarios and all 259 finite traces. A global-invalidation ablation emits no unsafe receipt in these suites, but completes only 6 of 11 benign targeted cases. A complete-descriptor equality control violates the historical-revocation contract in 4 targeted cases and 36 finite traces. The study contributes an executable contract, replayable counterexamples and controls that distinguish safety from useful completion. It establishes neither a new authorization primitive nor semantic attack-detection accuracy: the measured question is whether a local host preserves the operation it has already approved.**

***Index Terms*—agent security, Model Context Protocol, conformance testing, tool dispatch, time-of-check to time-of-use, reproducibility**

1. Introduction

Tool-using agents delegate operations to external services through a host. The Model Context Protocol (MCP) standardizes tool discovery and invocation, but a host still decides which server supplies a tool, which metadata reaches selection, when approval occurs and which request is sent. These decisions occur at different times. A successful metadata check is therefore only one event in a longer authorization lifecycle [1], [2].

Consider a host that exposes a file-reading tool from one server, approves a call, and later resolves the tool by its unqualified name. Another server can publish the same name between approval and dispatch. A request that was approved for the first server may reach the second. A different host might use a server-qualified name but retain mutable arguments; caller-side mutation after approval then changes the outbound request. A third host can snapshot both objects yet forget that an approval was revoked when its binding changed and was subsequently restored. Equal current bytes do not necessarily mean uninterrupted authorization.

These examples are host-contract failures, not evidence that a language model misunderstood a malicious instruction. Cryptographic tool identity, scoped capabilities, execution-time revalidation and reference monitors already address closely related problems [3]–[6]. A security evaluation must consequently avoid crediting a new gate for outperforming only a deliberately weak adapter. It must test the strongest appropriate conventional design and expose any utility cost of stricter invalidation.

We study a concrete selected-operation contract: after a host approves a tool call, an enqueue must use the same host-assigned server and connection, the same complete descriptor snapshot, and the same request, provided that the relevant binding has not been revoked. One approval can produce at most one enqueue. Updates unrelated to the selected operation should preserve that operation. These requirements are simple to state but span several interfaces that are often evaluated separately.

Our artifact makes the distinction observable. The tested gate emits bytes to an inert sink; a separate oracle compares the sink's records with detached approval records and declared revocation events. The oracle never asks the gate whether its own decision was correct. We evaluate six adapters sharing one content scanner: a weak integration control, a server-scoped wrapper using the upstream descriptor projection, complete snapshot equality, complete snapshots with revocation memory, a per-binding generation gate and a global-invalidation ablation.

The contribution is an executable empirical method and a bounded set of original results, not a claim that pre-dispatch mediation is new. Specifically, the artifact provides (1) a receipt-based contract separating correspondence, revocation, single-use and completion; (2) targeted and completely enumerated finite traces that distinguish content equality from authorization history; and (3) a reproducible comparison that includes a strong conventional snapshot control and publishes raw failures, benign invalidations and local timing samples. The strong control's success is a substantive result: it constrains what is necessary to achieve this contract and prevents attribution of safety to a particular counter representation.

The scope is intentionally narrower than overall agent security. No experiment uses an autonomous model, a live third-party service or a production credential. We do not estimate attack prevalence, detection recall, real-world exploitability or remote code integrity. We distinguish a source-level library behavior from a host-integration assumption throughout the paper.

2. Background and Related Work

2.1. Discovery, identity and invocation

MCP tool descriptors include a name and argument schema, with an optional description and further metadata fields. Names are scoped to server tool catalogues; a host with multiple connections must maintain its own association between a visible tool and the connection used to invoke it. List refreshes and cached tool definitions introduce lifecycle state beyond a single description string [1], [2], [7]. This study uses an explicit JSON-RPC fixture request and a fixed host contract. It is not a complete MCP SDK, protocol certification or survey of deployed hosts. The fixture's request shape is pinned to the basic 2025-11-25 name/arguments call envelope. It does not admit the newer request metadata, retry or header-projection fields discussed in the 2026-07-28 specification. Complete-request binding below means completeness over this declared supported shape, not every current MCP message field.

Tool-definition integrity and identity are established research topics. ETDI addresses tool squatting and rug pulls using identity and policy mechanisms. Origin-scoped signing and pinning appear in MCPS; the inspected draft's high-level description and its enumerated signing fields differ, so we do not assume it binds every possible descriptor extension [3], [8]. Authenticating a descriptor and maintaining the continuity of an already approved operation are related but different tasks. Our experiments grant the host trustworthy connection labels and ask what occurs as those labels or descriptor snapshots change.

2.2. Execution-time authority and agent reference monitors

Dynamic capabilities, host constraints and attested tool admission make execution-time checks central to their designs [4]–[6]. The Handle-Capability Protocol is especially close in proposing an invariant-oriented reference runtime and evaluation. Attested tool-server admission also checks admission at dispatch and observes effects in an evaluation harness. Work on agent time-of-check to time-of-use and verdict staleness examines validity at use time [9], [10]. Individual principal-binding and execution-finality drafts propose exact-action and sink-related authority bindings [11], [12]. Accordingly, the present implementation does not introduce the reference-monitor idea, capabilities, linearization or a new cryptographic trust anchor.

Our narrower unit of comparison is a host adapter receiving the same catalogue events and proposed operation. We isolate six choices: global versus server-qualified routing, descriptor projection versus full snapshots, equality versus revocation memory, mutable versus copied arguments, refresh completion order and selected-binding versus global invalidation. A strong snapshot implementation receives the same synchronization and full request boundary as the proposed generation gate. This comparison can reveal equivalence, not merely favorable ablations.

The closest studies already provide substantial evaluation machinery: HCP tests grant, handle and other runtime invariants with comparison arms and ablations; attested admission observes transport writes and permitted-call controls; the execution-finality draft specifies action/argument binding, revocation epochs and atomic use [5], [6], [12]. Our measured increment is the joint treatment of descriptor/request correspondence, restoration after revocation, refresh ordering and selected-operation completion in one local adapter contract. The matching results of immutable views and generation counters test whether that contract depends on its representation. We do not run those prior systems as experimental arms or infer comparative effectiveness under their different trust models. These are differences in evaluated scope, not claims that their implementations lack a defense.

2.3. Semantic security and evaluation

Descriptor-level manipulation and MCP safety benchmarks address malicious content and agent outcomes [13], [14]. They motivate protecting the tool boundary but do not make static metadata classification equivalent to safe dispatch. We reuse SENTINEL-TPD only as a pinned common content filter [15]. Its predictions are not the ground truth for lifecycle safety, and its author is also the author of this study. The content rules are frozen; the paper reports no new poisoning-detection percentage. Our synthetic data avoid borrowing external attack corpora with unresolved reuse terms.

Linearizability supplies the familiar vocabulary for making a concurrent operation appear to take effect at one instant [16]. We apply that idea to the trusted enqueue boundary and test selected orderings with barriers. Passing those tests is not a proof of all possible schedules. Similarly, enumerating a small event alphabet completely establishes coverage of that alphabet and depth, not coverage of all MCP behavior.

3. Threat Model and Contract

3.1. Assets, adversary and trusted components

The protected asset is the correspondence between an approved local operation and its actual enqueue. The trusted host owns the policy decision, connection identity, catalogue cache, approval store, gate and transport sink. A server may publish new descriptors, use names that collide with another server, withdraw a tool or reconnect. It cannot forge the host's internal connection identity or rewrite the gate and sink. The test driver also models caller-owned argument and descriptor mutation and out-of-order refresh completions. Those are explicit host-workflow conditions, not arbitrary adversarial access to trusted memory.

The host has already authorized the meaning of the call when it requests a permit. The reference gate's default structural validation does not infer user intent. Our comparison adapters install the same deterministic content policy, but an allowed description is not assumed benign. Authentication of a remote process, transport confidentiality, sandbox isolation, credential scoping, prompt-injection classification and unchanged-metadata malicious behavior are outside the claim.

3.2. Immutable records

Let an approval be

$$A = (s, c, n, D, Q), \tag{1}$$

where s is a host-assigned server identity, c a connection identity, n a tool name, D the detached complete descriptor, and Q the detached JSON-RPC request. The sink records an enqueue $E = (s', c', D', Q')$ independently of the gate's returned status. Under the structural equality relation $\equiv$, object-key order is irrelevant, arrays are ordered and scalar types are preserved. Non-finite numbers, duplicate raw JSON keys and invalid Unicode scalars are rejected by the oracle.

The correspondence invariant requires

$$s' = s \ \wedge \ c' = c \ \wedge \ D' \equiv D \ \wedge \ Q' \equiv Q. \tag{2}$$

The request includes the approved tool name and arguments, so matching a descriptor fingerprint alone cannot establish this invariant. A relevant revocation before enqueue invalidates the approval even if the descriptor later returns to its earlier value. An approval permits at most one enqueue. A failed or absent dispatch is not automatically a successful benign outcome.

3.3. Revocation and useful completion

The declared policy revokes an outstanding approval on selected-tool withdrawal, selected-descriptor change or connection change. Equivalent object-key reorderings, unrelated server updates and unrelated tool additions preserve the selected operation. A fresh approval can authorize recovery after a legitimate change. Refresh publication uses latest-requested-wins order: an older completion cannot overwrite a newer accepted catalogue. A refresh that occurs after an enqueue does not retroactively invalidate it.

These are the study's policy decisions. They are not requirements imposed on every MCP host. A host may legitimately use a stricter full-catalogue-context policy or continue using a last-good immutable descriptor under a different revocation contract. Our global-epoch arm measures one such stricter policy against selected-operation completion. Its additional blocks are availability costs under this contract, not evidence of a universally incorrect security design.

We record safety and declared completion separately. An unsafe case contains at least one receipt violating correspondence, revocation or single-use. A completion failure has a receipt count or content different from the declared case outcome. Blocking every request satisfies a vacuous no-unsafe-enqueue property but fails the ordinary completion controls. We publish both denominators.

4. Implementations and Common Adapter Boundary

4.1. Pinned content scanner

The common scanner is SENTINEL-TPD. We pin revision `5ce5d56e57a0acd092435fca5d89b78d0b39ee7b`. It scans configured metadata text with severity-tagged rules, computes a descriptor projection fingerprint and returns registration/invocation decisions. It does not itself provide a complete host or transport. Twelve vendored files retain their upstream license and exact Git blob and SHA-256 identities. The upstream regression suite is executed separately from our newly authored study tests.

4.2. Reference gate

The reference gate validates and detaches each descriptor, preserves all JSON fields, and stages a complete server catalogue before publication. Duplicate names within one server and unsupported descriptor shapes reject that publication. The validator checks bounded JSON and selected MCP field shapes; it is not a complete JSON Schema implementation. Admission sees a detached parse. Publication uses the original validated snapshot and name even if a trusted callback mutates its input.

Bindings are keyed by (s, n). Each binding retains a monotonically increasing generation across withdrawal, reappearance, descriptor change and connection change. Equivalent snapshots preserve the generation. At prepare, the gate checks that the selected binding remains current, freezes the complete request and issues an opaque, single-use local token. At execute, it consumes the token, rechecks the selected generation and invokes the trusted writer with the frozen connection and bytes. The lock spans revalidation and the writer's enqueue call.

The writer must enqueue the supplied bytes directly. It must not resolve a name again, re-enter the gate or wait for a remote response while retaining this lock. If the writer raises after sending, the gate reports an unknown transport outcome; it does not report a proved safe denial or make the token reusable. Production transports would need a bounded enqueue and separate response handling. The local study uses no general transport stack.

4.3. Why a strong snapshot control matters

The complete snapshot with revocation arm implements the same selected-operation contract without generation counters. It preserves an existing immutable view object across identical refreshes and replaces that view on a relevant change. A permit remains valid only while its original view is current. Thus a change-and-revert cannot resurrect an older view. It copies arguments and serializes check/enqueue just as the reference gate does. Their equivalence would show that monotone counters are one implementation choice, not a prerequisite for this safety property.

The equality-only arm deliberately lacks this history. It can compare a current full descriptor with an old full descriptor and find them equal after withdrawal and readdition. We label this as an ablation of the stated revocation contract, not an attack against an independently claimed capability of upstream software. The scoped-upstream and naive controls are likewise attributed to their explicit host adapters. This attribution prevents an artificial integration defect from being presented as a library vulnerability.

4.4. Argument for the serialized boundary

Under the trusted-writer assumption, a successful reference enqueue follows a token lookup and selected-

TABLE 1. COMPARISON ARMS. ALL HOST WRAPPERS ARE STUDY IMPLEMENTATIONS; ONLY THE VENDORED SCANNER/REGISTRY IS ATTRIBUTED TO UPSTREAM. NO ARM IS A DEPLOYED-PRODUCT BENCHMARK.

Arm	Bound state	Intended comparison
Naive host	Bare name; mutable arguments	Deliberately weak integration control; late routing and incremental publication.
Scoped upstream	Server/connection; upstream descriptor projection	Detached arguments and serialized dispatch; tests the effect of incomplete metadata and no historical revocation.
Snapshot equality	Server/connection; full descriptor and request	Conventional complete equality check; restoration can revive a prior view.
Snapshot revocation	Full immutable view plus invalidation memory	Strong separately implemented control; old view identity remains invalid after replacement.
Binding gate	Full snapshots plus per-binding generation	Reference mechanism; equivalent selected bindings preserve permits, relevant change invalidates them.
Global epoch	Binding gate plus any accepted refresh epoch	Stricter ablation that can invalidate permits after harmless catalogue activity.

binding comparison while publication is excluded by the same lock. The token contains detached bytes and is removed before the writer runs. Consequently, a concurrent accepted refresh occurs either before that comparison or after that enqueue; the former can invalidate the token and the latter cannot retroactively change the receipt. A replay finds no pending token. This local argument explains the intended implementation invariant. It assumes correct lock use, trustworthy connection identifiers and a writer honoring its contract; it does not establish distributed transactions, remote side-effect atomicity or correctness under arbitrary host compromise.

5. Study Design and Reproduction

5.1. Targeted scenarios

The first suite contains 36 declared scenarios: 24 single-property development cases and 12 composition-verification cases. Families cover namespaces, source aliasing, stale selection, description/schema/extension changes, withdrawal, connection replacement, change-and-revert, argument copying, replay, recovery and refresh order. Positive controls include normal calls, identical refreshes, reordered keys, unrelated updates and freshly approved recovery.

The composition labels are disjoint from development labels, but the author and coordinator could read their definitions while developing the implementation and runner. They are therefore specification-aware verification cases, not a blinded holdout or independent attack sample. The saved manifest and hashes establish the exact evaluated cases. We do not claim public preregistration.

5.2. Finite event model

The second suite enumerates every event word of length zero through three over six operations following one approval: publish the original descriptor on the original connection; publish a changed descriptor; withdraw the tool; publish the original on a second connection; update another server; and mutate caller-owned arguments. The count is $1 + 6 + 6^2 + 6^3 = 259$. Each trace ends with one execute. The expected state follows the declared contract without importing the gate, adapter or receipt oracle.

Once a relevant change revokes the original approval, restoring original bytes does not undo that revocation. Unrelated activity and argument mutation preserve the originally approved request. This model completely covers a small declared alphabet and bound. It omits parallel schedules, replay, reapproval, metadata extension fields and general request streams; complementary targeted and barrier tests cover selected examples of those behaviors. The two suites remain separate in the results and are never counted as independent attack families.

5.3. Receipt oracle and failure retention

Every arm receives fresh but equivalent fixture objects, including the original caller aliases needed for copy-boundary tests. The runner records an external, detached approval at the selection/prepare boundary. Reapproval replaces that record only when a valid new permit is issued. Each sink receipt carries the approval and relevant-revocation state at the instant of enqueue. Later events cannot rewrite a recorded result.

The oracle imports no gate or generation logic. It decodes sink bytes and compares typed structures and host identities. Independent negative controls cover server, connection, descriptor and request mismatch, duplicate keys, missing approvals, revocation, replay and a gate status that disagrees with an actual receipt. Errors are retained as errors, never converted into safe non-dispatch. Per-case rows preserve expected outcome, observed receipts, violations and completion.

5.4. Independent checks and development corrections

An adversarial review added scheduling-barrier tests for simultaneous replay and refresh attempts during a paused writer. It also reproduced two development gaps: an admission callback could mutate a parsed tool name before publication, and the oracle accepted an overflowing JSON number or lone Unicode surrogate. The former requires trusted callback misuse; it is not an ordinary remote-server bypass. Both were corrected before the recorded study. The report preserves pre-fix descriptions and distinguishes repaired verifier robustness from measured defense efficacy. Reentrant trusted writers remain outside the documented contract.

5.5. Local performance measurements

The primary run uses CPython 3.12.14 on an Intel Core i9-9880H MacBookPro16,1 with 16 GiB installed memory. Each of five method operations uses 20 warmup pairs and 200 measured pairs, with 16 operations per batch. A seed of 20261006 randomizes the six method orders within each pair. Adapter construction and precondition setup occur outside timing; method work and the in-memory sink append remain inside. The five boundaries are unchanged refresh, selection, preparation, execute-through-enqueue and complete refresh/select/prepare/execute integration. A shared-scanner measurement is separate. The run contains 6,820 timing rows, including warmups, and 99,200 measured method/scanner operations. Median and nearest-rank 95th percentiles summarize batch means, not single-request tails. OS, interpreter, timer details and the source-file hashes are saved with each run. We report absolute batch-mean quantiles and retain paired samples for reanalysis. The microbenchmarks exclude network latency, language-model inference and remote tool execution. Randomized method order reduces order bias but does not remove background load, thermal changes or interpreter effects. Memory and throughput claims are omitted because this study does not measure them. Repetition reduces timing noise for a fixed operation; it does not increase security-case diversity.

6. Results

6.1. Targeted conformance and completion

The saved primary run contains 216 targeted arm-case executions (36 cases across six methods) and 1554 separate finite-model executions. Every execution completed without a harness error; this status means the experiment ran, not that an arm satisfied the contract. Table 2 reports the actual unsafe-case and completion counts.

The naive integration control violates correspondence or revocation in 23 of 36 targeted cases. Server scoping and copied requests remove several failures, but the scoped upstream-projection adapter still violates nine cases. Complete-descriptor equality reduces this count to four; its remaining violations are restoration after relevant revocation. These are observed behaviors of the authored wrappers under the stated policy.

Both revocation-aware implementations complete every targeted case: 21 safe dispatches and 15 correct rejections each. Their 11 benign controls all complete. Global invalidation also has zero unsafe targeted cases, but completes only 25 of 36 declared outcomes, including six of 11 benign controls. It makes 26 safe rejections where the strong selected-binding implementations make 15. Safety-only reporting would hide those 11 additional completion failures.

6.2. Finite coverage and representative counterexamples

Of the 259 event words, 40 preserve the original approved operation and 219 revoke it. Both strong arms emit exactly 40 valid receipts and correctly reject the other 219. The equality-only and scoped-projection arms each produce 36 unsafe traces. This alphabet has no descriptor-extension event, so their equal finite-model outcomes do not contradict their different targeted-scenario outcomes. The global-epoch arm emits four valid receipts and rejects 255 traces, unnecessarily suppressing 36 of the 40 preserving traces.

For example, publication of a changed descriptor followed by publication of the original restores byte equality but not the revoked approval. The equality-only adapter enqueues the old request after this sequence; the revocation-aware arms reject it. Conversely, a refresh on another server followed by caller argument mutation should enqueue the original frozen request under this policy. A global epoch suppresses that otherwise valid request. The artifact preserves the event order, detached approval and actual receipt for each case.

The upstream-only observation program separately records 18 source-contract observations, including positive controls and explicitly conditional integration cases. All match the predicted observations. They are not counted as 18 attack families or pooled with either conformance denominator. The primary run also passes 75 authored tests and 17 upstream tests. Test counts establish the performed checks, not a probability of security.

6.3. Local cost

Table 3 reports primary-session quantiles of batch means in microseconds. The strong snapshot control has lower measured dispatch cost than the reference gate in this session, while their complete integration paths order differently. Different copy, validation, registry and logging work contributes to these values; the experiment does not isolate a universal cost of generations or establish statistical superiority.

Raw warmups, empty-harness controls, method orders and all timing samples are retained. They allow different summaries without reconstructing measurements from rounded table cells.

A separate process session on the same interpreter reproduces every deterministic targeted result. Its integration medians are 109.13 microseconds for the generation gate and 123.83 microseconds for the revocation-aware snapshot. The sessions are reported separately and are not treated as a broad sample of deployment conditions.

A fresh replay under Python 3.14.8 also reproduces every stored deterministic outcome for both suites. Timing tables remain restricted to the Python 3.12.14 sessions.

6.4. Representative pinned-library observations

Table 4 expands the separately reported source/API observations. It distinguishes a documented validation discrepancy from field-scope and host integration conditions. Ordinary covered-metadata mutation is denied, and a clean refresh restores admission as documented; that recovery is not a bypass.

The pinned source and each observation's code locations accompany the raw records; the schema-denial wording appears in the scanner's module documentation [15]. These observations explain which obligations

TABLE 2. MEASURED CONFORMANCE COUNTS. UNSAFE CASES CONTAIN AT LEAST ONE VIOLATING SINK RECEIPT. FULL COMPLETION REQUIRES BOTH SAFETY AND THE DECLARED OUTCOME. DENOMINATORS ARE DESIGNED CASES, NOT INDEPENDENT ATTACKS.

	36 targeted scenarios			259 finite traces	
Arm	Unsafe	Full completion	Benign completion	Unsafe	Full completion
Naive host	23/36	13/36	10/11	210/259	49/259
Scoped upstream	9/36	27/36	11/11	36/259	223/259
Snapshot equality	4/36	32/36	11/11	36/259	223/259
Snapshot revocation	0/36	36/36	11/11	0/259	259/259
Binding gate	0/36	36/36	11/11	0/259	259/259
Global epoch	0/36	25/36	6/11	0/259	223/259

TABLE 3. PRIMARY-SESSION LOCAL COSTS IN MICROSECONDS. QUANTILES ARE OVER 200 BATCH MEANS (16 OPERATIONS PER BATCH), NOT INDIVIDUAL-REQUEST TAIL LATENCY.

	Execute		Integration	
Arm	p50	p95	p50	p95
Naive host	28.67	35.85	133.75	159.65
Scoped upstream	44.83	53.88	213.32	239.18
Snapshot equality	1.91	2.18	126.89	149.01
Snapshot revocation	1.87	2.77	125.79	145.48
Binding gate	2.23	2.63	111.52	132.15
Global epoch	2.32	2.80	112.78	135.44

the study's host adapters add to the content API. They are not additional conformance cases or evidence of MCP SDK behavior.

6.5. Supplemental subprocess boundary check

Six additional checks connect the reference gate to two local child processes through standard-input/output JSON lines. The child echoes the exact received request bytes and records its own frame count and hashes at end-of-file. Normal calls and caller argument mutation deliver the original approved bytes; identical tool names on two connections reach their respective processes. Descriptor and connection replacement each produce zero received frames at both children. A single-use check produces one frame, and a fresh approval after reconnect reaches the new connection once. All six checks and nine transport regression tests pass. The last full replay under Python 3.14.8 passes 84 authored and 17 upstream tests.

This supplement checks an actual process boundary beyond the in-memory sink. It uses default structural admission and an authored, restricted call-envelope fixture; it does not compare content detectors or establish interoperability with an MCP SDK. It adds no cases to the 36/259 denominators and makes no timing claim.

7. Interpretation and Engineering Implications

The comparison separates three propositions. First, complete current descriptor equality is insufficient for a policy that remembers relevant revocation. Second, approved request contents and an unambiguous host identity must be preserved through enqueue. Third, invalidating all approvals on every catalogue refresh is stronger than the selected-operation contract and can discard useful approvals. These propositions follow from explicit event traces, not a claim about how frequently existing hosts encounter those traces.

A strong snapshot implementation can satisfy the same contract as per-binding generations. The practical engineering recommendation is therefore to make the contract explicit and test it at the transport boundary. A host can choose the representation that fits its architecture: object identity, monotonically tracked versions or another correctly synchronized revocation record. The artifact's tests are intended to survive such representation changes because the oracle does not check the implementation's internal counters.

The distinction between blocking and prevention remains essential. A status value can say deny even after a writer enqueues and raises. Conversely, a stricter arm can emit no unsafe receipt while denying ordinary useful work. Measuring the sink and expected completion prevents both outcomes from being summarized as perfect protection. The transport-error tests intentionally retain uncertainty where the host cannot know whether a side effect occurred.

8. Limitations and Threats to Validity

Synthetic and authored scope. The scenarios, wrappers, oracle and reference mechanism were constructed for one declared contract. They are not an ecosystem sample or independent attack dataset. The baseline library shares human authorship with the study. Separate code and adversarial checks reduce some implementation bias but are not independent human peer review. No statistical population confidence interval is meaningful for the designed trace counts, so none is reported.

Conditional policy and representation. Another host may authorize a different object or apply a different revocation policy. Full descriptor byte snapshots conservatively distinguish values that the structural oracle can treat as numerically equivalent, including signed zero. They also bind unknown extension fields even if a deployment does not display them. These choices can cause additional invalidations beyond the measured simple cases. The serialization is restricted Python JSON, not a cross-language canonicalization standard. The validation depth, node and size limits bound inputs but do not constitute full schema validation or denial-of-service protection.

Concurrency and integration. The finite words are serial. Barrier tests cover selected interleavings and the gate's lock discipline; the local subprocess supplement checks a restricted wire boundary. Neither covers all schedules or a production transport integration. The trusted writer must obey a non-reentrant enqueue contract; long-running writers can stall refreshes. Tokens and audit entries have no production lifecycle service, storage

TABLE 4. REPRESENTATIVE OBSERVATIONS OF THE PINNED SENTINEL-TPD LIBRARY, WITH CONTROLS FROM THE SAME OBSERVATION RUN. THESE ARE LOCAL API DECISIONS, NOT REMOTE EXECUTION OR HARMFUL OUTCOMES.

Boundary	Preconditions	Observed result and control
Schema validation	Raw dictionaries reach the library without SDK schema validation.	Null, string, number, list and missing `inputSchema` variants all pass scanning, registration and invocation checks, contrary to documented malformed-schema denial. An ordinary object schema also passes.
Descriptor projection	The host consumes a top-level `title`; covered fields stay unchanged.	Changing only the title to the controlled denial marker preserves the fingerprint and permits invocation. The same marker in the covered description denies registration and invocation. This is a projection-scope observation.
Registration ordering	A separate actor mutates a shared descriptor after scanning but before fingerprinting; a test audit hook only schedules the interleaving.	The changed descriptor passes invocation although a fresh scan denies it. Copying the descriptor before registration makes the changed current descriptor fail invocation. This condition assumes shared mutable host state; thread safety is not promised.
Server namespace	One unnamespaced registry serves two trusted connections with identically named tools.	Registering B overwrites A; checking A denies it and then B. Distinct registries with copied descriptors permit both. This is an availability collision, not malicious dispatch.

quota or distributed recovery design. Fixed request IDs are fixture-only.

Semantic and external validity. An unchanged descriptor can front malicious code. A correctly bound request can still be harmful if its initial approval was wrong. The experiments do not test LLM selection, user understanding, credential authority, transport authentication, untrusted tool responses or sandbox escapes. Local timings do not predict deployed latency. Results concern the pinned files and reported interpreter/hardware only.

Novelty boundary. Related work already studies identity, capabilities, execution-time checks, approval timing and reference runtimes. Our contribution is the specific executable conformance method, controls and bounded measurements. A dated, targeted source audit is included; it is not a proof that no similar test suite exists. A future independent multi-host study would strengthen external validity substantially.

9. Ethics, Harm and Data Availability

The study uses inert local descriptors and event records. It makes no requests to third-party targets, accesses no secrets, and executes no attack payloads. Reported library behaviors are paired with their documented integration assumptions; intended refresh recovery is not labeled a vulnerability. Publishing ambiguous counterexamples without those assumptions could unfairly characterize a library or encourage unsafe integrations, so the artifact retains controls and scope labels.

All experiment inputs are included and all study code is licensed for reuse. No external attack corpus or commercial model account is needed. The artifact provides vendored-source verification, exact manifests, raw JSONL outcomes, environment records, analysis programs, tests and a reproduction entry point. Timing measurements will vary across machines; deterministic outcome comparison is checked separately. The reviewer package is supplied as an offline supplementary archive; a public author repository is not required to reproduce the experiments. The package has no dependency on an unpublished external dataset or model service.

10. Conclusion

Approval-to-dispatch correspondence is an observable host property with several distinct failure modes. A content verdict alone does not specify that property. This study supplies a replayable contract and original bounded observations across six adapters, targeted cases, finite event words and local measurements. Its strong comparison avoids treating one internal representation as the source of security. The evidence supports testing identity, complete operation snapshots, revocation history, single-use and useful completion together at the enqueue boundary, while leaving semantic maliciousness and real deployment effectiveness to separate evaluations.

Acknowledgments and AI Assistance

OpenAI Codex assisted with literature retrieval and checking, implementation, fixture and test generation, analysis scripts, and drafting throughout this paper and its artifact. Exact API model snapshots were not recorded. The human author is responsible for reviewing the sources, code, results and text and for all submission declarations. No AI system is listed as an author, and no independent human peer review is claimed.

References

[1] Model Context Protocol, "Model Context Protocol: Tools", 2025, Specification version 2025-11-25; accessed October 6, 2026. https://modelcontextprotocol.io/specification/2025-11-25/server/tools.

[2] Model Context Protocol, "Model Context Protocol: Tools", 2026, Specification version 2026-07-28; accessed October 6, 2026. https://modelcontextprotocol.io/specification/2026-07-28/server/tools.

[3] Manish Bhatt, Vineeth Sai Narajala, Idan Habler, "ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP) by using OAuth-Enhanced Tool Definitions and Policy-Based Access Control", arXiv:2506.01333, 2025, Version v1, 2025-06-02; inspected preprint. https://arxiv.org/abs/2506.01333v1.

[4] Ziling Zhou, "Governing Dynamic Capabilities: Cryptographic Binding and Reproducibility Verification for AI Agent Tool Use", arXiv:2603.14332, 2026, Version v2, 2026-03-19; inspected preprint. https://arxiv.org/abs/2603.14332v2.

[5] Ting Liu, "From Tool Connection to Execution Control: Benchmarking Security Invariants in MCP-Style Agent Runtimes", arXiv:2606.29073, 2026, Version v1, 2026-06-27; inspected preprint. https://arxiv.org/abs/2606.29073v1.

[6] Alfredo Metere, “Attested Tool-Server Admission: A Security Extension to the Model Context Protocol”, arXiv:2605.24248, 2026, Version v1, 2026-05-22; inspected preprint. https://arxiv.org/abs/2605.24248v1.

[7] Model Context Protocol, “Model Context Protocol: Caching”, 2026, Specification version 2026-07-28; accessed October 6, 2026. https://modelcontextprotocol.io/specification/2026-07-28/server/utilities/caching.

[8] Raza Sharif, “MCPS: Cryptographic Security Layer for the Model Context Protocol”, Internet-Draft draft-sharif-mcps-secure-mcp-01, 2026, Individual submission, work in progress; revision 01, 2026-08-21. https://datatracker.ietf.org/doc/html/draft-sharif-mcps-secure-mcp-01.

[9] Derek Lilienthal, Sanghyun Hong, “Mind the Gap: Time-of-Check to Time-of-Use Vulnerabilities in LLM-Enabled Agents”, arXiv:2508.17155, 2025, Version v1, 2025-08-23; inspected preprint. https://arxiv.org/abs/2508.17155v1.

[10] Ilai Shraga, Roei Eshel, Lior Gorelik, “Approved Too Late: Verdict Staleness in LLM-Guarded Self-Adaptive Systems”, arXiv:2608.26306, 2026, Version v1, 2026-08-26; inspected preprint. arXiv DOI marked pending registration. https://arxiv.org/abs/2608.26306v1.

[11] Songbo Bu, “Security Principal and Verifier Binding for Agent Communication Protocols”, Internet-Draft draft-bu-agentproto-security-principal-binding-07, 2026, Individual submission, work in progress; revision 07, 2026-09-15. https://datatracker.ietf.org/doc/html/draft-bu-agentproto-security-principal-binding-07.

[12] Sangam Das, “Computation Is Not Authority: Hardware-Enforced Execution-Finality for Agentic AI, MCP Tool Calls, and Industrial Agents”, Internet-Draft draft-das-hardware-enforced-execution-finality-02, 2026, Individual submission, work in progress; revision 02, 2026-09-09. https://datatracker.ietf.org/doc/html/draft-das-hardware-enforced-execution-finality-02.

[13] Saeid Jamshidi, Arghavan Moradi Dakhel, Kawser Wazed Nafi, Foutse Khomh, “Semantic Attacks on Tool-Augmented LLMs: Securing the Model Context Protocol Against Descriptor-Level Manipulation”, arXiv:2512.06556, 2026, Version v2, 2026-05-21; inspected preprint. https://arxiv.org/abs/2512.06556v2.

[14] Dongsen Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, Wenjun Xu, “MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents”, arXiv:2510.15994, 2026, Version v2, 2026-03-24; inspected preprint. https://arxiv.org/abs/2510.15994v2.

[15] Rajeshkumar Sampathrajan, “sentinel-tpd”, Software repository, Revision 5ce5d56e57a0acd092435fca5d89b78d0b39ee7b. Accessed October 6, 2026. https://github.com/rajesamp/sentinel-tpd/tree/5ce5d56e57a0acd092435fca5d89b78d0b39ee7b.

[16] Maurice P. Herlihy, Jeannette M. Wing, “Linearizability: A Correctness Condition for Concurrent Objects”, *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990. https://www.cs.cmu.edu/~wing/publications/HerlihyWing90.pdf.